클래스의 상속

클래스의 상속

클래스의 상속 (Inheritance)

기존에 정의되어 있는 클래스를 이용하여 변수와 메소드를 추가하고 새로운 클래스를 정의하는 것

클래스의 상속

클래스의 상속이란?

"상속"의 의미처럼 사용됨

- 이미 정의되어 있는 클래스를 상속하면 상속을 하는 클래스의 멤버(변수, 메소드)들을 물려받음
- 새로운 클래스는 상속하는 멤버들을 자신의 멤버로 사용함

기존에 정의되어 있던 클래스	상속을 통해 새롭게 정의되는 클래스
조상 클래스 또는 상위 클래스, 기초 클래스	자식 클래스 또는 하위 클래스, 파생 클래스

클래스의 상속

클래스 상속의 목적

1 코드의 재사용

- 프로그래밍 작업의 생산성을 높이고, 유지 보수와 같은 변경사항에 효율적으로 대처 가능
- 상속은 코드의 재사용을 목적으로 함

2 상속을 통한 새로운 클래스 구성 시 가이드 역할 제공

- 필요한 변수, 메소드의 규칙을 적용하는 것

클래스의 상속

상속의 관계 설정

개념적으로 넓은 의미를 가지고 있는 클래스가 조상 클래스로 정의하여 공통적인 것들을 정의

클래스 상속의 기본 문법

```
// 조상 클래스, 상위 클래스, 기본 클래스
class Parent {
}
// 자손 클래스, 하위 클래스, 새로 만드는 클래스
```

```
class Child extends Parent {
}
```

새로 만드는 자손 클래스의 경우 상속을 하기 위해서는 클래스 선언부에 extends 키워드를 사용해서 조상 클래스를 지정

상속을 하게 되면 조상 클래스의 변수들과 메소드들이 새로운 클래스로 상속되지만 생성자는 상속되지 않음

클래스 상속의 기본 문법

```
package chapter07;
public class Phone {
	String phoneNumber;
	void call(){
		System.out.println(phoneNumber + "에서 전화를 걸어 통화를 합니다. ");
	}
}
package chapter07;
public class SmartPhone extends Phone { // Phone 클래스를 상속
	String model;
	void game(){
		System.out.println(model + " 게임을 합니다.");
	}
}
```

클래스 상속의 기본 문법

```
package chapter07;
public class InheritanceTest {
	public static void main(String[] args) {
		SmartPhone smartPhone = new SmartPhone();
		// 조상 클래스로 부터 상속받은 변수
		smartPhone.phoneNumber = "010-0000-0000";
		// 자식 클래스에서 추가 로 정의한 변수
		smartPhone.model = "Google";
		// 조상 클래스로 부터 상속받은 메소드
		smartPhone.call();
		// 자식 클래스에서 추가 로 정의한 메소드
		smartPhone.game();
	}
}
```

IS~A

클래스 간의 관계를 결정하기 위해서는 “IS~A” 관계나 혹은 “HAS A”관계가 성립이 되는지 확인해야 함

상속의 관계로 정의하기 위해서는 “IS~A”관계가 성립되어야 함

상위 클래스로 정의할 클래스가 다른 하위 클래스들의 대표성을 가진다면 “IS~A”관계가 됨

예

스마트폰은 전화기 이다. → 스마트폰 is 전화기

유선전화기는 전화기 이다. → 유선전화기 is 전화기

→ 전화기가 대표성을 가지기 때문에 전화기는 상위 클래스로 정의하고 스마트폰과 유선전화기는 하위 클래스로 정의할 수 있음

HAS A

HAS~A 관계는 클래스 간의 대표성을 가지지 않지만 다른 클래스의 멤버를 사용해야 하는 경우 정의

```
public class Point {
    int xPos;
    int yPos;
}
```

```
public class Line {
    //int point1Xpos;
    //int point1Ypos;
    //int point2Xpos;
    //int point2Ypos;
    Point point1;
    Point point2;
}
```

"선은 점이다, 선 is 점"과 같이 "IS~A"관계가 성립 되지는 않지만
선은 점을 포함하고 있기 때문에 이러한 관계는 "HAS~A"관계로 표현

상위 클래스의 멤버 호출

COMPILER DESIGN
Reinhard Wilhelm · Dieter Maurer
COMPILER DESIGN
Java
C++

변수 구분, 메소드 호출

상속 관계에 있는 클래스에서 멤버들의 구분은 super 키워드를 이용해 구분함

super 키워드는 상위 클래스를 지정하는 용도로 사용

하위 클래스에서 상위 클래스의 변수를 참조하거나 메소드를 호출할 때에 super 키워드를 이용

변수 구분

```
package chapter07;
public class Parent {
	int num = 10;
}
```

```
package chapter07;
public class Child extends Parent {
	int num=100;
	void showData(){
		System.out.println("조상 클래스의 변수 num=" + super.num);
		System.out.println("자손 클래스 인스턴스 변수 num=" + this.num);
	}
}
```

변수 구분

```
package chapter07;

public class InheritanceTest2 {

	public static void main(String[] args) {

		Child child = new Child();

		child.showData();

	}
}
```

상위 클래스의 메소드 호출

```
package chapter07;
public class Phone {
        String phoneNumber;
        public Phone() {
        }
        public Phone(String phoneNumber) { // phoneNumber 변수를 초기화 하는 생성자
                this.phoneNumber = phoneNumber;
        }
        void call(){
                System.out.println(phoneNumber + "에서 전화를 걸어 통화를 합니다. ");
        }
}
```

상위 클래스의 메소드 호출

```
package chapter07;
public class SmartPhone2 extends Phone {
        String model;
        public SmartPhone2() {}
        public SmartPhone2(String phoneNumber, String model) {
                super(phoneNumber);
                this.model = model;
        }
        void game(){
                super.call(); // 상위 클래스의 call()메소드를 호출
                System.out.println(model + " 게임을 합니다.");
        }
}
```

상위 클래스의 메소드 호출

```
package chapter07;
public class InheritanceTest3 {
	public static void main(String[] args) {
		SmartPhone2 smartPhone = new SmartPhone2();
		// 조상 클래스로 부터 상속받은 변수
		smartPhone.phoneNumber = "010-0000-0000";
		// 자식 클래스에서 추가 로 정의한 변수
		smartPhone.model = "Google";
		// 자식 클래스에서 추가 로 정의한 메서드
		smartPhone.game();
	}
}
```

생성자 호출

인스턴스 생성 시 멤버의 초기화

- 상속관계에서 생성자는 상속의 대상이 아니기 때문에 인스턴스 생성시에 멤버의 초기화는 하위 클래스의 생성자가 상위 클래스의 생성자를 호출해서 상위 클래스의 멤버 변수를 초기화

하위 클래스의 생성자는 상위 클래스에서 초기화 할 멤버의 값을 받아서 상위 클래스의 생성자를 호출해야 함

상위 클래스의 생성자 호출도 상위 클래스의 멤버를 호출할 때 사용하는 키워드 super를 사용

생성자 호출

```
package chapter07;
public class Phone {
	String phoneNumber;
	public Phone() {
	}
	public Phone(String phoneNumber) { // phoneNumber 변수를 초기화 하는 생성자
		this.phoneNumber = phoneNumber;
	}
	void call(){
		System.out.println(phoneNumber + "에서 전화를 걸어 통화를 합니다. ");
	}
}
```

생성자 호출

```
package chapter07;
public class SmartPhone extends Phone { // Phone 클래스를 상속
        String model;
        public SmartPhone() {
        }
        public SmartPhone(String phoneNumber, String model) {
                super(phoneNumber);  // 상위 클래스의 생성자 호출
                this.model = model;
        }
        void game(){
                System.out.println(model + " 게임을 합니다.");
        }
}
```

생성자 호출

```
package chapter07;
public class InheritanceTest4 {
	public static void main(String[] args) {
		SmartPhone smartPhone = new SmartPhone("010-7777-7777", "IOS");
		// 조상 클래스로 부터 상속받은 메소드
		smartPhone.call();
		// 자식 클래스에서 추가 로 정의한 메소드
		smartPhone.game();
	}
}
```

생성자 호출

상속을 이용해서 새롭게 정의한 클래스의 인스턴스 생성과정

먼저 상위 클래스와 하위 클래스의 생성자가 메모리에 로드 됨

↓

SmartPhone 클래스의 생성자가 호출

↓

SmartPhone 클래스 생성자의 super키워드로 상위 클래스 Phone 클래스의 생성자를 호출

↓

초기화 된 후 SmartPhone 클래스의 생성자에서 나머지 변수의 초기화

생성자 호출

상속을 이용해서 새롭게 정의한 클래스의 인스턴스 생성과정

```
new SmartPhone("010-7777-7777", "IOS");  // 1. 하위 클래스의 생성자 호출

public SmartPhone(String phoneNumber, String model) {
  super(phoneNumber);        // 2. 상위 클래스의 생성자 호출
  this.model = model;         // 4. 하위 클래스에서 변수 초기화
}

public Phone(String phoneNumber) {
  this.phoneNumber = phoneNumber;   //3. 상위 클래스의 변수 초기화
}
```

메소드의 재구성 : 오버라이딩

오버라이딩

메소드의 오버라이딩

상속받은 방을 원하는 데로
변경해서 사용하듯이 상속
관계에 있는 자손 클래스가
조상 클래스의 멤버인
메소드를 재구성하는 것

오버라이딩

재구성

상위 클래스에 정의된
메소드의 이름, 반환 타입,
매개변수선언의 메소드
선언부가 완전히 동일한
메소드를 메소드 처리
블록 안의 문장들로 새롭게
구성하는 것

오버라이딩

오버라이딩 된 경우 상위 클래스의 메소드는 오버라이딩 된 메소드에 의해 가려짐

상위 클래스의 메소드가 호출되는 것이 아니라 오버라이딩 된 메소드가 호출됨

오버라이딩

오버라이딩을 하기 위한 조건

1. 상속 관계에서 상위 클래스에서 상속한 메소드를 하위 클래스에서 재구성하는 경우
2. 메소드의 반환타입, 이름, 매개변수의 정의 선언부가 동일해야 함
3. 접근제어자에 의해 큰 범위로 변경은 가능하지만 작은 범위로의 변경은 불가함
4. 상위 클래스의 메소드에서 정의한 예외 타입보다 많은 예외를 선언할 수 없음

오버라이딩

```
package chapter07;
public class SmartPhoneImpl extends Phone {
        String model;
        public SmartPhoneImpl(String phoneNumber, String model) {
                super(phoneNumber); // 상위 클래스 생성자 호출
                this.model = model;
        }
        @Override
        void call() {  // 상위 클래스의 메소드를 재구성
                super.call(); // super 키워드는 상위 클래스를 가르키는 키워드
                System.out.println("이어팟을 이용해서 통화를 합니다.");
        }
```

오버라이딩

```
        public static void main(String[] args) {
                SmartPhoneImpl phone = new SmartPhoneImpl("010-9999-0000", "IOS");
                phone.call();     // 오버라이딩된 메서드가 호출됩니다.
        }
}
```

어노테이션 @Override

어노테이션은 우리가 사용하는 JAVA의 문법은 아니지만 주석과 같은 기능이 없는 것이 아닌
추가적인 의미나 기능을 가지고 있음

```
package chapter07;
public class SmartPhoneImpl extends Phone {
        String model;
        public SmartPhoneImpl(String phoneNumber, String model) {
                super(phoneNumber); // 상위 클래스 생성자 호출
                this.model = model;
        }
        @Override
        void call() {  // 상위 클래스의 메소드를 재구성
                super.call(); // super 키워드는 상위 클래스를 가르키는 키워드
                System.out.println("이어팟을 이용해서 통화를 합니다.");
```

메소드가 오버라이딩 하고 있으니 오버라이딩의 규칙에 맞게 잘 정의가 되었는지 체크하는 기능을 가지고 있으며, 오버라이딩 하는 메소드가 오버라이딩 규칙에 어긋난 경우 오류 표현

다형성

COMPILER DESIGN
Reinhard Wilhelm · Dieter Maurer
COMPILER DESIGN
Java
C++

다형성

다형성(Polymorphism)이란 여러 가지 형태를 가진다는 의미

JAVA에서는 조상 클래스의 한 타입 중 참조변수로 자손 클래스의 여러 타입의 인스턴스를 참조할 수 있도록 하여 프로그램을 구현하는데 사용됨

다형성을 이용한 참조변수를 선언하고 인스턴스를 생성

```
조상 클래스 변수이름 = new 자손 클래스 이름();
```

다형성

```java
package chapter07;
public class SmartPhoneImpl2 extends Phone {
	String model;
	public SmartPhoneImpl2(String phoneNumber, String model) {
		super(phoneNumber); // 상위 클래스 생성자 호출
		this.model = model;
	}
	@Override
	void call() { // 오버라이딩
		super.call(); // super 키워드는 상위 클래스를 가리키는 키워드
		System.out.println("이어팟을 이용해서 통화를 합니다.");
	}
	void playMusic() { // 새로운 메소드를 정의
		System.out.println("다이너마이트가 플레이됩니다.");
	}
}
```

다형성

```
package chapter07;
public class SmartPhoneMain {
	public static void main(String[] args) {
		Phone phone = new SmartPhoneImpl2("010-1111-2222", "Android");

		phone.call(); // 상위 클래스에 정의되어 있기 때문에 호출가능.
				// 하위 클래스의 오버라이딩 된 메소드가 호출됨.
		//phone.playMusic(); // 상위 타입의 참조변수이기 때문에 메소드 사용불가
	}

}
```

다형성

조상 클래스 타입의 참조변수로 선언했는데, 상위 클래스의 메소드가 실행한 것이 아니라
하위 클래스에서 정의한 메소드가 호출됨

참조변수를 상위 클래스로 정의했기 때문에 상위 클래스인 Phone 타입의 멤버만 사용이 가능

Phone
String phoneNumber
void call()

SmartPhoneImpl2
String phoneNumber
void call()
void playMusic()

```
Phone phone = new SmartPhoneImpl2();
phone.phoneNumber="010-0000-0000" // 사용가능
phone.call(); // 사용가능 오버라이딩된 메소드 호출
phone.playMusic(); // 오류, 인스턴스에는 있지만 참조변수의 타입에서 정의하지 않음
```

다형성을 이용한 형 변환

참조변수 phone이 phone.playMusic(); 메소드를 사용하기 위해서는 참조변수 phone을 SmartPhoneImpl2 타입의 변수로 자료형 변환을 해주어야 함

이때 형 변환 연산자를 이용해서 자료형을 변환

참조형 변수는 상속의 관계에서 자료형 변환 연산자를 이용하여 자료형 변환을 할 수 있음

상속관계에서의 형 변환의 특징

- 하위 클래스 타입에서 상위 클래스 타입으로의 자료형 변환
  → 형 변환 연산이 생략 가능함
- 상위 클래스 타입에서 하위 클래스 타입으로의 자료형 변환
  → 명시적으로 형 변환 연산자를 이용해서 형 변환을 해야 함

다형성을 이용한 형 변환

```
package chapter07;
public class AndroidPhone extends Phone {
	public AndroidPhone(String phoneNumber) {
		super(phoneNumber); // 상위 클래스 생성자 호출
	}
}
```

```
package chapter07;
public class IPhone extends Phone {
	public IPhone(String phoneNumber) {
		super(phoneNumber); // 상위 클래스 생성자 호출
	}
}
```

다형성을 이용한 형 변환

```
package chapter07;
public class ClassTypeChange {
    public static void main(String[] args) {
        AndroidPhone androidPhone = new AndroidPhone("010-0000-0000");
        Phone phone1 = (Phone) androidPhone; // 형 변환 연산자를 이용해서 형 변환
        phone1.call();
        Phone phone2 = androidPhone; // 형 변환 연산자 없이 자동 형 변환
        phone2.call();
    }
}
```

다형성을 이용한 형 변환

```
package chapter07;
public class ClassTypeChange2 {
        public static void main(String[] args) {
                AndroidPhone androidPhone = new AndroidPhone("010-0000-0000");
                IPhone iPhone = new IPhone("010-9999-9999");
                Phone phone1 = androidPhone;
                Phone phone2 = iPhone;
                //IPhone iPhone1 = phone2;  // 오류
                IPhone iPhone1 = (IPhone) phone2; // 형 변환
                IPhone iPhone2 = (IPhone) phone1;  // 오류
        }
}
```

자료형 타입 확인

잘못된 자료형 변환은 오류를 발생 시킴

실제로 참조하고 있는 인스턴스 타입을 확인하고 형 변환할 필요가 있음

- 다형성으로 인해 참조변수가 실제 참조하고 있는 인스턴스 타입이 아닌 상위 클래스 타입의 변수로 사용되기 때문

참조변수가 어떤 자료형의 인스턴스를 참조하고 있는지 확인을 할 때는 특정 자료형 타입으로 형 변환이 가능한지를 판별해주는 instanceof 연산자를 이용

참조변수 instanceof 클래스이름

- 왼쪽의 참조변수가 오른쪽에 기술한 클래스로 형 변환이 가능한지 판별하고 true 또 는 false 논리값을 반환

다형성을 이용한 형 변환

```
package chapter07;
public class ClassTypeChange3 {
        public static void main(String[] args) {
                AndroidPhone androidPhone = new AndroidPhone("010-0000-0000");
                IPhone iPhone = new IPhone("010-9999-9999");

                Phone phone1 = androidPhone;
                Phone phone2 = iPhone;

                IPhone iPhone1 = null;
                // 변수 phone2이 IPhone 타입으로 형 변환 가능 여부 판별
                if(phone2 instanceof IPhone) {
                        iPhone1 = (IPhone) phone2;
                        iPhone1.call();
                } else {
                        System.out.println("자료형 변환이 불가합니다.");
                }
```

다형성을 이용한 형 변환

```
                IPhone iPhone2 = null;
                // 변수 phone1이 IPhone 타입으로 형 변환 가능 여부 판별
                if(phone1 instanceof IPhone) {
                        iPhone2 = (IPhone) phone1;
                        iPhone2.call();
                } else {
                        System.out.println("자료형 변환이 불가합니다.");
                }
        }
}
```

매개변수의 다형성

메소드의 선언할 때 매개변수를 정의하는데 이때 받아야 하는 타입이 여러 개인 경우 각 타입에 맞는 여러 개의 메소드가 정의되어야 함

매개변수의 타입들이 상속관계에 있다면 이 매개변수를 상위 타입의 변수로 정의하고, 만들어야 하는 메소드의 개수를 줄여 효율적으로 정의할 수 있음

```
AndroidPhone androidPhone = new AndroidPhone("010-0000-0000");
IPhone iPhone = new IPhone("010-9999-9999");

callByPhone(androidPhone);
callByPhone(iPhone);

// Phone 클래스를 확장하는 클래스가 생성되면 메소드도 같이 정의해주어야 합니다.
void callByPhone(IPhone phone){
    phone.call();
}

void callByPhone(AndroidPhone phone){
    phone.call();
}
```

```
AndroidPhone androidPhone = new AndroidPhone("010-0000-0000");
IPhone iPhone = new IPhone("010-9999-9999");

callByPhone(androidPhone);
callByPhone(iPhone);

// 매개변수의 다형성 활용
// Phone클래스를 확장하는 클래스가 생성되어도 메소드를 추가로 정의하지 않아도 됩니다.

void callByPhone(Phone phone){
    phone.call();
}
```

자료형 타입 확인

```
package chapter07;
public class ClassTypeChange4 {
        public static void main(String[] args) {
                AndroidPhone androidPhone = new AndroidPhone("010-0000-0000");
                IPhone iPhone = new IPhone("010-9999-9999");
                callByPhone(androidPhone);
                callByPhone(iPhone);
        }
        // 전화를 거는 매소드 : Phone 타입의 인스턴스 참조 값을 받는다.
        public static void callByPhone(Phone phone){
           phone.call();
        }
}
```

다형성을 이용한 객체의 배열 처리

배열을 사용하는 가장 큰 가치는 여러 개의 변수를 묶어 일괄처리를 할 수 있다는 것

다양한 상속관계의 확장 클래스들을 묶어 사용할 수 있다면 데이터 처리가 효율적일 것

다형성을 이용하면 상속관계에서 서로 다른 하위 클래스들의 인스턴스들을
하나의 배열로 묶어 사용하는 것이 가능

→ 다형성의 특징인 상위 클래스 타입의 변수로 정의할 수 있기 때문

다형성을 이용한 객체의 배열 처리

```
package chapter07;
public class Application {
        String type;
        String appName;
        String company;
        int size;
        public Application(String type, String appName, String company, int size) {
                this.type = type;
                this.appName = appName;
                this.company = company;
                this.size = size;
        }
        public void AppRun() {
                // 새로운 Application 클래스에서 실행에 대한 정의
        }
```

다형성을 이용한 객체의 배열 처리

```
        public void stop() {
                System.out.println(appName + " 애플리케이션을 종료합니다.");
        }
        public void dislayAppInfo() {
                System.out.println("-----------------------");
                System.out.println("애플리케이션 종   류 : " + type);
                System.out.println("애플리케이션 이   름 : " + appName);
                System.out.println("애플리케이션 개발사 : " + company);
                System.out.println("애플리케이션 사이즈 : " + size);
                System.out.println("-----------------------");
        }
        @Override
        public String toString() {
                return "Application [type=" + type + ", appName=" + appName + ", company=" +
company + ", size=" + size + "]";
        }
}
```

다형성을 이용한 객체의 배열 처리

```
package chapter07;

public class GameAppImpl extends Application {

	public GameAppImpl(String appName, String company, int size) {
		super("GAME", appName, company, size);
		// TODO Auto-generated constructor stub
	}

	@Override
	public void AppRun() {
		System.out.println(">>>>>>>>>>> " + appName + " 게임을 합니다.");
	}
}
```

다형성을 이용한 객체의 배열 처리

```
package chapter07;

public class UtilAppImpl extends Application {

	public UtilAppImpl(String appName, String company, int size) {
		super("UTIL", appName, company, size);
	}

	@Override
	public void AppRun() {
		System.out.println(">>>>>>>>>>> " + appName+" 유틸을 실행해서 사진을
편집합니다.");
	}
}
```

다형성을 이용한 객체의 배열 처리

```
package chapter07;
public class SmartPhoneImpl3 extends Phone {
        String model;
        GameAppImpl[] gameApps;
        UtilAppImpl[] utilApps;
        int gameAppCnt = 0;
        int utilAppCnt = 0;
        public SmartPhoneImpl3(String phoneNumber, String model) {
                super(phoneNumber);
                this.model = model;
                gameApps = new GameAppImpl[100];
                utilApps = new UtilAppImpl[100];
        }
```

다형성을 이용한 객체의 배열 처리

```
        // 게임 앱만 설치 하는 메서드
        void installApp(GameAppImpl app) {
                gameApps[gameAppCnt++] = app;
                System.out.println(app.appName + " 앱을 설치했습니다.");
        }
        // 유틸 앱만 설치하는 메서드
        void installApp(UtilAppImpl app) {
                utilApps[utilAppCnt++] = app;
                System.out.println(app.appName + " 앱을 설치했습니다.");
        }
        @Override
        void call() {
                super.call(); // super 키워드는 상위 클래스를 가르키는 키워드
                System.out.println("이어팟을 이용해서 통화를 합니다.");
        }
}
```

다형성을 이용한 객체의 배열 처리

SmartPhoneImpl3 클래스의 문제

설치되는 앱의 종류가 증가할 때마다 설치된 앱을 저장하는 배열과, 설치된 앱의 개수를 저장하는 변수, 설치하는 메소드가 추가로 정의되어야 함

- 다형성성의 특징을 이용해서 배열을 정의하면 하나의 배열과, 하나의 변수, 하나의 메소드만으로 구현이 가능해 짐

다형성을 이용한 객체의 배열 처리

```
package chapter07;
public class SmartPhoneImpl4 extends Phone {
        String model;
        Application[] installedApps; // 상위 클래스타입의 배열 하나만 정의
        int cntOfApps;
        public SmartPhoneImpl4(String phoneNumber, String model) {
                super(phoneNumber);
                this.model = model;
                installedApps = new Application[100];
                cntOfApps=0;
        }
        // 다형성을 이용하면 다양한 타입의 앱을 받아서 처리할 수 있다.
        void installApp(Application app) {
                installedApps[cntOfApps++]=app;
                System.out.println(app.appName+" 앱을 설치했습니다.");
        }
```

다형성을 이용한 객체의 배열 처리

```
// Application 타입의 배열과 찾고자 하는 앱의 이름을 받아 애플리케이션 객체를 반환
Application getApplication(Application[] storeApps, String name) {
        Application app = null;
        for(int i=0; i<storeApps.length; i++) {
                if(storeApps[i].appName.equals(name )) {
                        app = storeApps[i];
                }
        }
        return app;
}
```

다형성을 이용한 객체의 배열 처리

```
        Application getIstalledApp(String name) {
                Application app = null;
                for(int i=0; i<cntOfApps; i++) {
                        if(installedApps[i].appName.equals(name)) {
                                app = installedApps[i];
                        }
                }
                return app;
        }
}
```

다형성을 이용한 객체의 배열 처리

```
package chapter07;
public class SmartPhoneMain2 {
        public static void main(String[] args) {
                // Phone 인스턴스 생성
                SmartPhoneImpl4 phone = new SmartPhoneImpl4("010-0000-0000",
"MyPhone");
                // 설치할 수 있는 애플리케이션 배열 생성과 초기화
                Application[] store = new Application[5];
                store[0] = new GameAppImpl("지뢰찾기", "MicroSoft", 1000);
                store[1] = new UtilAppImpl("깨똑", "카카5", 5000);
                store[2] = new GameAppImpl("프리쉘", "MicroSoft", 3100);
                store[3] = new UtilAppImpl("너튜브", "Google", 5000);
                store[4] = new UtilAppImpl("인별그램", "Facebook", 1000);
               // 폰에 유틸 애플리케이션 너트뷰 설치
                //phone.installApp(store[3]);
```

다형성을 이용한 객체의 배열 처리

```
                Application app = phone.getApplication(store, "너튜브");
                if(app!=null) {
                        phone.installApp(app);
                } else {
                        System.out.println("스토어에 찾으시는 애플리케이션이 없습니다.");
                }
                // 너튜브 앱 실행
                Application installedApp = phone.getIstalledApp("너튜브");
                if(installedApp!=null) {
                        installedApp.dislayAppInfo();
                        installedApp.AppRun();
                }
        }
}
```